

## บทที่ 2

# ทฤษฎีพื้นฐานเกี่ยวกับเว็บเซอร์วิส

### 2.1. เว็บเซอร์วิส

เว็บเซอร์วิส เป็นระบบซอฟต์แวร์ที่ออกแบบมาเพื่อสนับสนุนการทำงานระหว่างคอมพิวเตอร์กับคอมพิวเตอร์ผ่านระบบเครือข่าย โดยที่ภาษาที่ใช้ในการติดต่อสื่อสารระหว่างคอมพิวเตอร์คือภาษาเอ็กซ์เอ็มแอล (XML)

การอธิบายวิธีการใช้งานของเว็บเซอร์วิสนั้นจะอธิบายโดยใช้ภาษา WSDL (Web Services Description Language) ซึ่งเป็นภาษา XML ประเภทหนึ่ง

ระบบอื่นๆ จะสามารถติดต่อและทำงานกับเว็บเซอร์วิส โดยใช้ โปรโตคอลที่ชื่อว่า SOAP (Simple Object Protocol) ซึ่งใช้ภาษา XML เป็นมาตรฐานในการติดต่อระหว่างระบบโดยผ่านทางโปรโตคอลอื่นที่ใช้ในการส่งข้อมูลบนเว็บ อาทิเช่น โปรโตคอล HTTP

#### 2.1.1. ประโยชน์ของการทำงานร่วมกันระหว่าง แอปพลิเคชัน กับ เว็บเซอร์วิส

เว็บเซอร์วิส ช่วยให้การเข้าถึงข้อมูลสารสนเทศจากแอปพลิเคชันที่ต่างกันเป็นไปโดยง่าย โดยแอปพลิเคชันนั้นๆ สามารถเขียนด้วย Java และรันอยู่บน Sun Solaris Application Server หรืออาจจะเขียนด้วย C++ และรันอยู่บน Windows NT หรืออาจจะเขียนด้วย Perl และรันอยู่บนเครื่อง Linux ซึ่งมาตรฐานของเว็บเซอร์วิส ทำให้อินเทอร์เน็ตเฟสของแอปพลิเคชันเหล่านี้ ถูกอธิบายโดย WSDL และทำให้อยู่ในมาตรฐานของ UDDI หลังจากนั้น จึงสามารถติดต่อสื่อสารถึงกันโดย XML ผ่าน SOAP อินเทอร์เน็ต

เว็บเซอร์วิส สามารถถูกเรียกใช้ภายในองค์กรเองหรือจากภายนอกองค์กร โดยผ่านไฟร์วอลล์ ดังนั้นจึงมีองค์กรใหญ่ๆ มากมาย กำลังพัฒนาระบบที่มีอยู่ของตน ให้เข้ากับ เว็บเซอร์วิส ซึ่งนับเป็นการลงทุนที่คุ้มค่า เนื่องจาก เว็บเซอร์วิส สามารถเพิ่มศักยภาพในการทำงานขององค์กร อีกทั้งลดค่าใช้จ่ายในการจัดการทรัพยากรขององค์กรได้อีกทางหนึ่ง

นอกจากนั้น เว็บเซอร์วิส ยังสามารถใช้ร่วมกับเว็บแอปพลิเคชัน โดยส่งผ่านข้อมูลทางอินเทอร์เน็ตได้อีกด้วยซึ่งนับเป็นวิธีที่มีประสิทธิภาพในการติดต่อสื่อสารกับลูกค้าหรือหุ้นส่วน ถึงแม้จะต้องคำนึงถึงระบบรักษาความปลอดภัย และการจัดการรายการของข้อมูลอยู่ก็ตาม แต่เว็บเซอร์วิส ได้ใช้มาตรฐานทั่วไปของ Internet เรื่องดังกล่าวจึงนับเป็นเรื่องธรรมดาของการสื่อสารผ่านระบบอิเล็กทรอนิกส์

### 2.1.2. การทำงานของเว็บเซอร์วิส

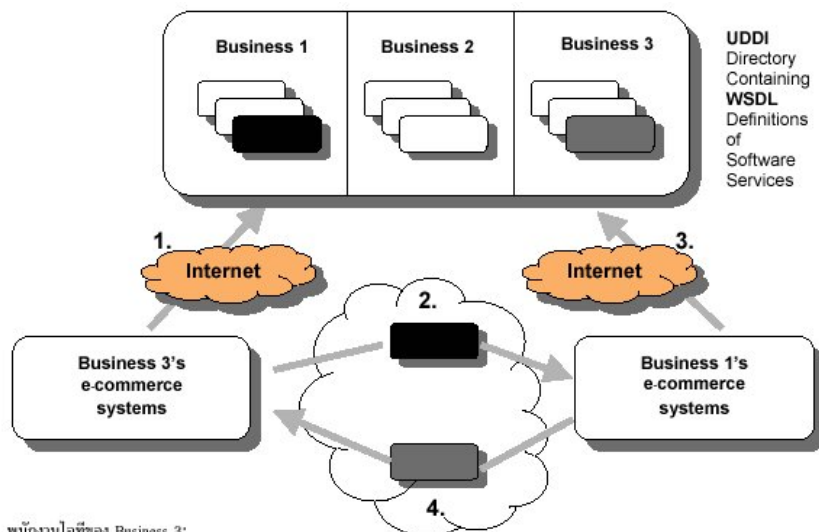
การทำงานของเว็บเซอร์วิสประกอบไปด้วยมาตรฐานหลัก 4 อย่างซึ่งอธิบายได้ง่าย ๆ ดังนี้

**1. XML (Extensible Markup Language)** เป็นภาษามาตรฐานที่ทุกระบบสนับสนุนทำให้ข้อมูลที่มีโครงสร้างของภาษา XML จะถูกนำไปประมวลผลต่ออย่างอัตโนมัติได้อย่างง่ายดาย ภาษามาตรฐาน XML จึงถูกนำมาใช้เป็นมาตรฐานในการแลกเปลี่ยนข้อมูลของ Web Services

**2. SOAP (Simple Object Access Protocol)** หรือโซฟ เป็นมาตรฐานของเทคโนโลยี Distributed Objects แบบหนึ่ง โดยทำหน้าที่ส่งข้อมูลผ่านอินเทอร์เน็ต ในรูปแบบของ XML ทำให้เรียกใช้งานโปรแกรมข้ามระบบผ่านทางอินเทอร์เน็ตได้

**3. WSDL (Web Services Description Language)** เป็นภาษามาตรฐานที่ใช้สำหรับอธิบายการใช้งานโปรแกรมที่เปิดให้บริการ ซึ่งเขียนขึ้นตามแบบมาตรฐาน XML ดังนั้น WSDL จึงเป็นเสมือนคู่มือให้กับระบบ เพื่อเรียนรู้วิธีการเรียกใช้งาน Web Services

**4. UDDI (Universal Description, Discovery, and Integration)** เป็นระบบมาตรฐานในการอธิบายและค้นหา Web Services โดยเป็นตัวกลางให้ Provider มาลงทะเบียนไว้ โดยใช้ไฟล์ WSDL บอกรายละเอียดของบริษัทและบริการที่มีให้ ทำให้ Requestor สามารถค้นหาและทราบว่าบริษัทมีผลิตภัณฑ์และบริการอะไรบ้าง สามารถติดต่อขอดำเนินธุรกิจการค้ากับบริษัทได้โดยอัตโนมัติผ่านทาง Web Services



พนักงานไอทีของ Business 3:

1. ค้นหาข้อกำหนดการให้บริการใน WSDL ผ่านอินเทอร์เน็ตของ Business 1
2. ติดต่อผ่านอินเทอร์เน็ตของแอปพลิเคชันที่ให้บริการ (ด้วย SOAP) ผ่านทางอินเทอร์เน็ต
- 3,4. Business 1 รวมหรือบูรณาการกับ Business 3 ด้วยวิธีเดียวกัน

รูปที่ 2.1 อธิบายการทำงานของเว็บเซอร์วิส

จากมาตรฐานทั้ง 4 อย่างที่กล่าวข้างต้นสามารถสรุปลำดับขั้นตอนการทำงานของ Web Services ได้ดังนี้

1. Provider จัดทำระบบหรือบริการที่เป็น Web Services ขึ้นมา
2. ทำการลงทะเบียน Web Services กับหน่วยงานที่ให้บริการระบบ UDDI (หรือ Registry)
3. นำ WSDL ไปใส่ไว้ในระบบ UDDI ที่ได้ลงทะเบียนไว้
4. Requestor ทำการค้นหาระบบหรือบริการที่ต้องการจากระบบ UDDI
5. เมื่อ Requestor ได้พบระบบหรือบริการที่ต้องการจะนำไฟล์ WSDL ไปเรียนรู้วิธีการเรียกใช้ผ่านระบบของตน
6. Requestor ทำการติดต่อและเรียกใช้ระบบหรือบริการจาก Provider ได้โดยตรงผ่าน SOAP ในระบบของตน

### 2.1.3. เปรียบเทียบระหว่าง Web Application และ Web Services

เมื่อเข้าใจความหมายและการทำงานของ Web Application และ Web Services แล้ว จะเห็นว่าเครื่องมือทั้งสองต่างใช้ HTTP Protocol หรือ อินเทอร์เน็ต เป็นช่องทางในการสื่อสารเหมือนกัน แต่มีวัตถุประสงค์ต่างกัน โดย Web Application ใช้เพื่อการแลกเปลี่ยนไฟล์ HTML ระหว่างเว็บเซิร์ฟเวอร์ แต่ Web Services เป็นการแลกเปลี่ยน “บริการ” (ก้อนโปรแกรม Software Components) ระหว่างระบบสารสนเทศผ่านเว็บเซิร์ฟเวอร์

ในเรื่องของความสามารถโดยส่วนใหญ่จะใช้ Web Application ในการติดต่อกับผู้ใช้ผ่านทาง Internet browser เพื่อนำเสนอข้อมูลและการทำงานหรือธุรกรรมต่างๆ ส่วน Web Services จะทำหน้าที่ในการติดต่อกับเว็บเซิร์ฟเวอร์เพื่อแลกเปลี่ยนข้อมูลและการทำงานหรือใช้บริการข้ามระบบกันโดยใช้ Web Application หรือ Application Interface ในการติดต่อกับผู้ใช้ นอกจากนี้ Web Services ยังสามารถทำงานกับระบบต่างๆ ได้มากกว่า 1 ระบบ ในขณะที่ Web Application ไม่สามารถทำได้โดยตรง ซึ่งสามารถสรุปการเปรียบเทียบได้ดังตาราง

ตารางที่ 2.1 แสดงการเปรียบเทียบระหว่าง เว็บแอปพลิเคชันกับเว็บเซอร์วิส

| หัวข้อ              | Web Services               | Web Applications           |
|---------------------|----------------------------|----------------------------|
| การเชื่อมต่อ        | program-program            | human-program              |
| ภาษาที่ใช้          | XML                        | HTML                       |
| รายชื่อการให้บริการ | ค้นหาผ่าน UDDI             | ค้นหาผ่าน search engine    |
| ขอบเขตการใช้งาน     | Business-to-Business (B2B) | Business-to-Customer (B2C) |
| โปรโตคอล(Protocol)  | SOAP+HTTP                  | HTTP                       |

หลังจากที่ได้ทำความเข้าใจความหมายและข้อเปรียบเทียบระหว่าง Web Application และ Web Services แล้ว ความแตกต่างระหว่างเครื่องมือทั้งสองจะเห็นได้ชัดในเรื่องของความสามารถ แต่ในการนำไปใช้งานจำเป็นจะต้องประยุกต์ใช้เครื่องมือทั้งสองชนิดร่วมกัน เพื่อให้เกิดประสิทธิภาพมากที่สุด

#### 2.1.4. การประยุกต์ใช้งานเว็บเซอร์วิส

ขณะเดียวกัน เว็บเซอร์วิส เป็นพื้นฐานสำคัญของการบริการใหม่ๆ ในลักษณะที่ต้องการเชื่อมโยง ระบบสารสนเทศที่มีความแตกต่างเข้าด้วยกันอย่างอัตโนมัติ เช่น การพาณิชย์อิเล็กทรอนิกส์ระหว่างธุรกิจกับธุรกิจ (B2B), ระบบการบริการรัฐบาลอิเล็กทรอนิกส์ ในลักษณะที่มีการเชื่อมบริการของภาครัฐกับภาครัฐเข้าด้วยกัน เพื่อทำให้เกิดการบริการต่อประชาชนจากจุดเดียวได้ (One Stop Services) เป็นต้น

ตัวอย่างพื้นฐานหนึ่ง คือ การเชื่อมโยงระหว่างระบบการสั่งซื้อ ของบริษัทผู้ขายบริษัทหนึ่ง ไปยังระบบสารสนเทศการ จัดส่งของบริษัทจัดส่งอีกบริษัทหนึ่ง จะเป็นไปได้ง่ายมากขึ้นด้วยเทคโนโลยีของเว็บเซอร์วิส พื้นฐานดังกล่าวนี้ จะทำให้ "การจัดการห่วงโซ่อุปทานด้วยสื่ออิเล็กทรอนิกส์" (e-SCM: Supply Chain Management) และธุรกิจผ่านสื่ออิเล็กทรอนิกส์ที่มีประสิทธิภาพ และมีความคุ้มค่ามากขึ้น โดยมีการแลกเปลี่ยนข้อมูลธุรกิจ ระหว่างองค์กรในห่วงโซ่อุปทาน

ทั้งนี้แต่ละบริษัทจะมีระบบงาน (แอปพลิเคชัน) และใช้รูปแบบของข้อมูล (ฟอร์มเมตดาต้า) ที่แตกต่างกันได้ ซึ่งเมื่อมีการส่งข้อมูลจากองค์กรหนึ่งไปอีกองค์กรหนึ่ง ระบบคอมพิวเตอร์ของบริษัทผู้รับ สามารถนำข้อมูล ที่ได้ไปประมวลผลต่อได้ทันที การเชื่อมโยงระบบอีคอมเมิร์ซของหน้าร้าน กับระบบหลังร้าน แล้วยังเชื่อมโยง กับระบบสารสนเทศของพันธมิตรทางการค้านี้ จะทำให้การทำธุรกิจผ่านสื่ออิเล็กทรอนิกส์ มีความคุ้มค่า มีประสิทธิภาพ และสร้างผลกำไรให้กับบริษัทได้

#### 2.2. Service-Oriented Architecture (SOA)

แนวคิดหลักของธุรกิจผ่านสื่ออิเล็กทรอนิกส์ คือกลยุทธ์การเพิ่มประสิทธิภาพของการดำเนินธุรกิจโดยใช้ อินเทอร์เน็ต อินทราเน็ตและเอ็กซ์ทราเน็ตในทุกระดับอย่างต่อเนื่อง

ปัจจุบันการพัฒนาระบบธุรกิจผ่านสื่ออิเล็กทรอนิกส์อาจจะใช้สถาปัตยกรรมการบริการในลักษณะที่เรียกว่า "Service-Oriented Architecture" (SOA) เป็นแนวคิดเบื้องต้น ระบบและแอปพลิเคชันส่วนใหญ่ในโลกของธุรกิจที่ใช้งานในปัจจุบันเป็น Application

SOA มีส่วนประกอบหลักสามส่วนคือ

- ผู้ให้บริการ (Service provider)
- ผู้ขอบริการ (Service requester)
- ตัวแทนของผู้ให้บริการ (Service broker)

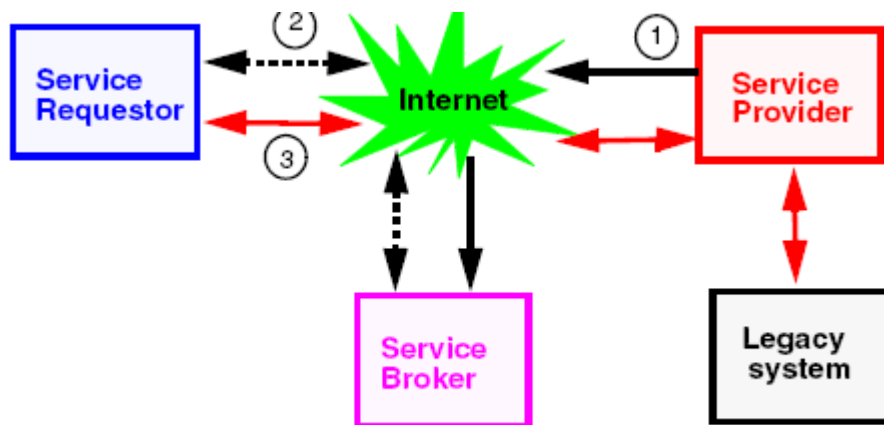


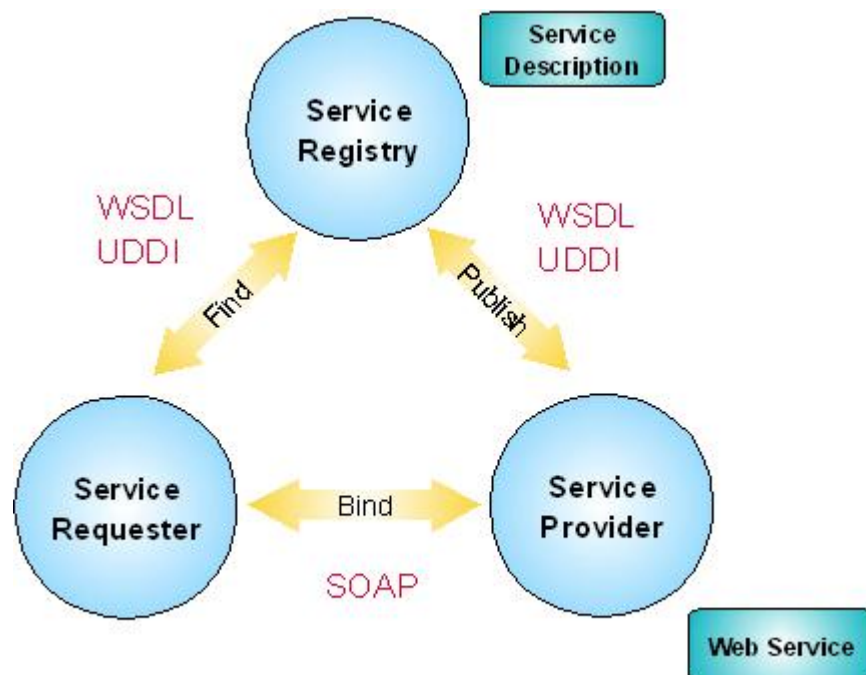
Figure 1-1 Web services roles and operations

## รูปที่ 2.2 แสดงส่วนประกอบและการทำงานของ SOA

ซึ่งส่วนประกอบหลักทั้ง 3 ส่วนนี้ติดต่อกันโดยใช้ฟังก์ชันพื้นฐาน คือ

- การประกาศ (publish)
- การค้นหา (find)
- การเรียกใช้ (bind)

ฟังก์ชันทั้งสามมีการทำงานดังนี้คือ ผู้ให้บริการ (Service provider) ทำการประกาศ (publish) บริการไปยังตัวแทนของผู้ให้บริการ (Service broker) หรือที่อาจเรียกว่า "ไดเรกทอรีของบริการ" ในขณะที่ผู้ขอบริการ (Service requester) จะทำการค้นหา (find) บริการที่ต้องการ และเมื่อพบเห็นก็จะทำการเรียกใช้ (bind) ไป



## รูปที่ 2.3 แสดงการติดต่อกันของส่วนประกอบหลัก

- ข้อดี SOA

เนื่องจากสถาปัตยกรรม SOA ของเว็บเซอร์วิสทำให้สามารถนำเอาโค้ดที่เขียนไปแล้วกลับมาใช้ใหม่ซึ่งไม่เพียงทำให้ประหยัดเวลาในการพัฒนาแต่ยังช่วยประหยัดต้นทุนส่วนเพิ่มทั้งในเรื่องฮาร์ดแวร์และซอฟต์แวร์ได้อีกด้วย

## 2.3. XML

XML (The Extensible Markup Language 1.0) เป็นภาษา Markup ที่เป็น text-based ซึ่งทำให้เป็น มาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตอย่างรวดเร็ว ผู้ที่ทำหน้าที่รับผิดชอบ และกำหนดมาตรฐานของ XML คือ World Wide Web Consortium (W3C) ความแตกต่างระหว่าง XML กับ HTML คือ HTML ถูกนำมาใช้ในการสร้าง เว็บเพจ ที่สามารถแสดงผลได้โดยโปรแกรมเบราว์เซอร์ แต่ XML จะใส่ tags ได้อย่างอิสระ แล้วทำการส่ง XML ชุดนี้ไป ประมวลผลยังแอปพลิเคชันใด ๆ ที่สามารถใช้ข้อมูลใน XML นี้

### 2.3.1. XML & HTML

HTML ภาษาที่ใช้ในการเขียน Web มากที่สุดนั่นเป็นเพราะมีรูปแบบที่ง่ายต่อการแสดงผลของ Browser เนื่องจาก มี tag ตายตัวที่สามารถบอกได้ว่าเมื่อเจอ tag นี้จะแสดงผลอย่างไร เช่น เมื่อเจอ tag <B>...</B> ในเอกสารก็ให้แสดงข้อความที่อยู่ระหว่าง tag เป็นตัวหนา แต่จะสังเกตได้ว่าคอมพิวเตอร์จะไม่เข้าใจว่าข้อความนั้นคืออะไร เพียงแต่รู้ว่าแสดงผลอย่างไร นั่นแสดงว่าไม่สามารถนำข้อมูลภายใน tag เหล่านี้ไปทำการประมวลใดๆ ได้เลย

XML เป็นภาษาที่มีลักษณะเป็น tag คล้าย HTML แต่ไม่ได้มุ่งที่การแสดงผล XML มุ่งที่การสื่อความหมายโดยอนุญาตให้ผู้ใช้สามารถกำหนด tag ขึ้นได้เองเพื่อให้สื่อความหมายทางภาษาของมนุษย์ แต่คอมพิวเตอร์เองก็เข้าใจเช่นกัน ทำให้ข้อมูลระหว่าง tag สามารถนำไปประมวลผลต่อได้ เช่น

```
<ComputerBook>
  <book>
    <name>เว็บเซอร์วิส</name>
    <price>10.00$</price>
  </book>
  <book>
    <name>xml</name>
    <price>10.00$</price>
```

```
</book>
</ComputerBook>
```

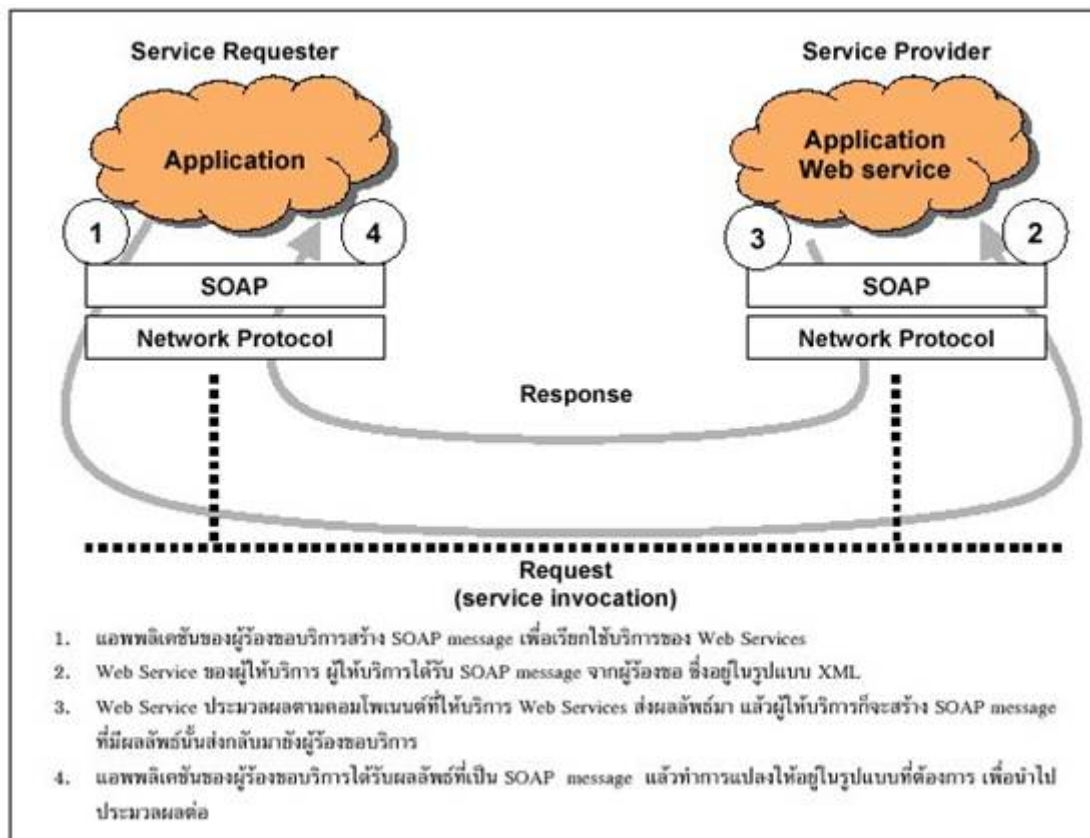
รูปที่ 2.4 แสดง Tag ของภาษา XML

จากตัวอย่าง จะเห็นได้ว่ารูปแบบเอกสาร xml ไม่ได้บอกวิธีแสดงผลไว้ แต่เอกสารสามารถสื่อความหมายให้คอมพิวเตอร์เข้าใจได้ และนำค่าไปประมวลผลต่อได้ แต่ถ้าเราต้องการจะแสดงผลเราต้องใช้ควบคู่กันกับHTML

## 2.4. SOAP (Simple Object Access Protocol)

SOAP กลายเป็นสิ่งที่มีความสำคัญสำหรับ Web Services อย่างรวดเร็ว เป็นโปรโตคอลที่ผู้จัดหา Web Services เลือกใช้ที่จะส่ง message ระหว่าง Web Services SOAP เป็น Transport Protocol ที่มี XML เป็นพื้นฐานและใช้ HTTP เป็นโปรโตคอลร่วมในการส่งผ่านเครือข่าย SOAP จะระบุวิธีในการเข้ารหัสส่วนหัว (Header Encoding) ของทั้ง HTTP และไฟล์ XML ให้อย่างชัดเจนทั้งใน ส่วนของการติดต่อไปยังคอมพิวเตอร์เครื่องหนึ่งและส่งผ่านข้อมูลไปให้ รวมถึงระบุวิธีที่โปรแกรมซึ่งถูกเรียกนั้นจะส่งค่าคืนกลับมาด้วย SOAP (Simple Object Access Protocol) เป็น XML-based โปรโตคอล (lightweight protocol) และใช้ HTTP เป็นโปรโตคอลร่วม สำหรับ การแลกเปลี่ยนข้อมูลในสภาวะแวดล้อมแบบกระจายศูนย์ (decentralized, distributed environment)

SOAP ได้ กำหนดเมสเสจิงโปรโตคอล (Messaging Protocol) ระหว่างผู้ขอบริการ (requestor) กับผู้ให้บริการ (provider) เช่น ผู้ขอบริการสามารถติดต่อแลกเปลี่ยนข้อมูลกับผู้ให้บริการโดยใช้ RMI (Remote Method Invocation) ตามวิธีการของ โปรแกรมแบบออปเจกต์ บริชท์ ไมโครซอฟท์, ไอบีเอ็ม, โลตัส, ยูสเซอร์แลนด์ (UserLand) และ ดีเวลลอปเปอร์เมนเตอร์ (DeveloperMenter) ได้ร่วมกันกำหนดมาตรฐานของ SOAP ขึ้น ซึ่งต่อมาได้มีบริษัทอีก 30 กว่า บริษัทเข้าร่วมและ จัดตั้งเป็น W3C XML Protocol Workgroup ขึ้น SOAP ได้กำหนดรูปแบบพื้นฐานของการสื่อสารแบบกระจายขึ้นโดย การพัฒนา SOA แม้ว่า SOA จะไม่ได้กำหนดเมสเสจิงโปรโตคอล (Messaging Protocol) ไว้ แต่ SOAP ได้ถูกกำหนด ให้เป็น Services-Oriented Architecture Protocol เรียบร้อยแล้ว เนื่องจากมันได้ถูกใช้ในการพัฒนา SOA อย่างแพร่ หลายแล้วนั่นเอง จุดเด่นของ SOAP ก็คือเป็นโปรโตคอลที่เป็นกลาง กล่าวคือ ไม่มีใครเป็นเจ้าของและเป็นโปรโตคอล ที่ทำงานกับโปรโตคอลอื่นหลายชนิด การพัฒนาก่อนุญาตให้ทำได้อย่างอิสระตามแพลตฟอร์มระบบปฏิบัติการ แบบจำลองทางวัตถุ (Object model) และภาษาโปรแกรมของผู้ที่ทำการพัฒนา



รูปที่ 2.5 แสดง XML messaging using SOAP

#### 2.4.1. ทำไมต้องเป็น SOAP

เนื่องจากจุดประสงค์หลักของการใช้งานเว็บเซอร์วิส เราต้องการให้แอปพลิเคชัน มีการทำงานกับแอปพลิเคชันที่ทำงานอยู่ในเครื่องอื่นโดยผ่านทางเครือข่าย ซึ่งเทคโนโลยีที่มีอยู่ในปัจจุบันที่ใช้มีการสื่อสารระหว่าง Object ในระยะไกล (Remote Procedure Calls : RPC) เช่น DCOM, EJB หรือ CORBA นั้นไม่ได้ถูกออกแบบมาใช้สำหรับโปรโตคอล HTTP (เครือข่ายที่ปัจจุบันใช้งานแพร่หลาย และสะดวกมากที่สุดได้แก่เครือข่าย Internet ซึ่งให้การสื่อสารผ่านโปรโตคอล HTTP เป็นส่วนใหญ่)

เทคนิค RPC ของเทคโนโลยีที่กล่าวข้างต้นนั้นต่างก็มีปัญหาในด้านการนำมาใช้งานในแง่ของความเข้ากันได้ของการเรียกใช้งานข้ามเทคโนโลยี เนื่องจากเป็นเทคโนโลยีเฉพาะของแต่ละค่าย ยกเว้น CORBA ผู้พัฒนาระบบจะต้องพัฒนาโปรแกรมที่มีความซับซ้อน และยังมีปัญหาในส่วนของ Firewalls และ Proxy Server ด้วยเนื่องจากโดยปกติ เซอร์เวอร์จะปิดการสื่อสารที่ไม่ใช่โปรโตคอล HTTP ออกไป เพื่อความปลอดภัยของระบบที่มีการติดต่อสื่อสารกับภายนอก

ดังนั้นทางเลือกของการสื่อสารที่จะนำมาใช้ในการให้บริการเว็บเซอร์วิส ก็คือให้ทำงานอยู่บน โปรโตคอล HTTP ไปเลย ซึ่ง SOAP นอกจากจะทำงานบนโปรโตคอล HTTP แล้วยังเป็น

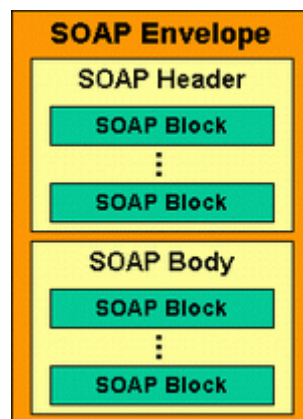


มาตรฐานเปิดที่จะทำให้สามารถติดต่อกับเครื่องคอมพิวเตอร์ที่มีความแตกต่างกันทั้งระบบปฏิบัติการ, เทคโนโลยี รวมไปถึงภาษาที่ใช้ในการพัฒนาด้วยก็ได้

#### 2.4.2. โครงสร้างของ SOAP

เอกสาร SOAP นั้นมีโครงสร้างในรูปแบบ XML ซึ่งเราสามารถแบ่งเป็นส่วนของเอกสารได้เป็น 3 ส่วนหลักดังนี้คือ

1. SOAP envelop เนื้อหาสาระ (Content) ของเอกสารทั้งหมด
2. SOAP header ส่วนเพิ่มเติมของเอกสาร SOAP ซึ่งจะมีก็ได้ หรือไม่มีก็ได้
3. SOAP body ส่วนที่ใช้ในการเรียกใช้งานเซอร์วิส และผลลัพธ์ที่ได้จากเซอร์วิส



รูปที่ 2.6 โครงสร้างของเอกสาร SOAP

```
<soap:Envelope>
  <soap:Body>
    <GetPrice>
      <Item>Rose</Item>
      <Quantity>100</Quantity>
    </GetPrice>
  </soap:Body>
</soap:Envelope>
```

รูปที่ 2.7 ตัวอย่างเอกสาร SOAP อย่างง่ายของการสอบถามราคา ดอกกุหลาบ จำนวน 100 ดอก

อีกตัวอย่างหนึ่งของเอกสาร SOAP ที่จะแสดงในรูปด้านล่างเพื่อจะได้ทำความเข้าใจในส่วนต่างๆ ที่มีความซับซ้อนขึ้น แต่ก็อย่าจนเกินไปที่จะทำความเข้าใจได้ดัง รูปที่ 2.8 SOAP Envelope ต่อไปนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"  
  
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">  
  <soap:Header>  
    <!-- ข้อมูลในส่วนของ Header -->  
    <i:local xmlns:i="http://www.i3t.or.th/ws/">  
      <i:currency>Bath</i:currency>  
    </i:local>  
  </soap:Header>  
  <soap:Body>  
    <!-- ข้อมูลในส่วนของ Body -->  
    <GetPrice>  
      <Item>Rose</Item>  
      <Quantity>100</Quantity>  
    </GetPrice>  
  </soap:Body>  
  <soap:Fault>  
    <!--ข้อมูลของ SOAP ในกรณีมีข้อผิดพลาด จาก SOAP Node -->  
  </ soap:Fault>  
</soap:Envelope>
```

รูปที่ 2.8 ตัวอย่างเอกสาร SOAP

ตารางที่ 2.2 แสดงส่วนประกอบของ SOAP

| Element    | Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <Envelope> | <p>จะเป็น Root ของเอกสาร XML SOAP เสมอ โดยจากตัวอย่างจะใช้ Namespaces จาก "http://schemas.xmlsoap.org/soap/envelope/" (Schema for the SOAP/1.1 envelope)</p> <p>ดังนั้น Element ต่างๆ ในตัวอย่างนี้ที่อยู่ในเอกสาร XML ส่วนที่เป็นมาตรฐานของ SOAP เราจะมี Namespaces ที่เราตั้งเป็น soap (&lt;soap:Envelop&gt;, &lt;soap:Header&gt;, &lt;soap:Body&gt;, &lt;soap:Fault&gt;)</p> <p>Attribute ใน Envelop</p> <p>Attribute encodingStyle ในตัวอย่างมีการอ้างการ encoding จาก "http://schemas.xmlsoap.org/soap/encoding/" (Schema for the SOAP/1.1 encoding)</p> <p>จะใช้ในกรณีที่เรามีการใช้ Parameter หลายชนิดข้อมูลในเอกสาร (Data Type) เช่น Boolean, String, Integer เป็นต้น</p>                                                                                                                   |
| <Header>   | <p>จากตัวอย่างมีการเพิ่มเนื้อหาของเอกสาร SOAP เข้าไปในส่วนของ Header ในที่นี้คือ สกุลเงิน (Currency) ซึ่งส่วนนี้จะใช้ในส่วนของการ Application ไม่ได้เป็นส่วนมาตรฐานของ SOAP แต่ผู้ใช้งานเป็นผู้กำหนดใช้เอง (User-defined)</p> <p>Attribute ใน Header</p> <p>ใน &lt;Header&gt; Element อาจจะมีการใส่ Attribute mustUnderstand เพื่อให้ฝั่งที่รับเอกสาร SOAP ให้ทำการประมวลผลในส่วน Header ด้วยโดยกำหนดค่าเป็น Boolean เช่น</p> <pre> &lt;i:local xmlns:i="http://www.i3t.or.th/ws/"     &lt;i:country mustUnderstand="1"&gt;Thailand&lt;/i:country&gt;     &lt;i:currency mustUnderstand="0"&gt;Bath&lt;/i:currency&gt; &lt;/i:local&gt; </pre> <p>ค่าของ mustUnderstand ถ้าเป็น "0" แสดงว่าไม่ต้องประมวลผล<br/>ถ้าเป็น "1" : true คือจำเป็นต้องมีการประมวลผล<br/>(โดยค่า default = "0" : fault)</p> |
| <Body>     | โดยปกติแล้วเอกสาร SOAP จะต้องมีส่วนของ Body เพราะเป็นส่วน                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

|         |                                                                                                                                                                                                                                                                                                                                   |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         | เป็นเนื้อหาสาระจริงๆ ของ SOAP จากตัวอย่างเราต้องการสอบถามราคาของดอกกุหลาบจำนวน 100 ดอก ซึ่ง <GetPrice>, <Item> และ <Quantity>เป็น Element ที่ใช้งานใน Application ไม่ใช่มาตรฐานของ SOAP                                                                                                                                           |
| <Fault> | เป็นส่วนที่จะถูกใช้เมื่อมีข้อผิดพลาดในการประมวลผลของเอกสาร SOAP ซึ่งโดยปกติจะเห็นเฉพาะที่เป็นเอกสารตอบกลับเท่านั้น (Reply Message หรือ Answer Message) ซึ่งตัวอย่างจะเป็นเอกสารที่เป็นการร้องขอบริการ (Request Message) ซึ่งไม่น่าจะมีส่วนของ <Fault> Element แต่ที่เขียนไว้เพื่อให้เห็นภาพรวมของโครงสร้างเอกสาร SOAP ทั้งหมดก่อน |

```

<soap:Fault>
  <faultcode>Client</faultcode>
  <faultstring>Invalid Request</faultstring>
</soap:Fault>

```

**รูปที่ 2.9** ตัวอย่างข้อมูลใน <Fault> Element ที่มีการเกิดข้อผิดพลาด

### ตารางที่ 2.3 อธิบาย Fault Element

| Sub Element   | คำอธิบาย                                   |
|---------------|--------------------------------------------|
| <faultcode>   | ชื่อของการเกิดข้อผิดพลาด (ดูตารางด้านล่าง) |
| <faultstring> | คำอธิบายของข้อผิดพลาด                      |
| <faultactor>  | ชื่อของแหล่งที่เกิดข้อผิดพลาด              |
| <detail>      | ข้อมูลรายละเอียดเพิ่มเติมของข้อผิดพลาดนั้น |

ตารางที่ 2.4 แสดง Fault code Error

| Error<br>(faultcode) | คำอธิบาย                                                                                                                     |
|----------------------|------------------------------------------------------------------------------------------------------------------------------|
| VersionMismatch      | มีข้อผิดพลาดของการอ้าง Namespace ของ Element                                                                                 |
| MustUnderstand       | ในการประมวลผลของ Sub Element ใน Header ที่มีการกำหนด attribute mustUnderstand เป็น "1" ไม่สามารถประมวลผลได้ (not understood) |
| Client               | เอกสาร SOAP มีข้อผิดพลาดอันเนื่องมาจากมีโครงสร้างทางผิด หรือมีข้อมูลที่ไม่ถูกต้อง                                            |
| Server               | มีข้อผิดพลาดจาก Server ไม่สามารถประมวลผลเอกสารได้                                                                            |

จะเห็นว่า SOAP Envelope ก็เป็นเอกสารที่อยู่ในรูปแบบ XML ทั่วไป แต่สิ่งที่ทำให้ SOAP มีความสามารถมากขึ้นก็คือ SOAP ได้มีการกำหนดโครงสร้างของเอกสารเป็นส่วนๆ โดยอธิบายว่าส่วนใดมีหน้าที่อะไร (What) มีข้อมูลอะไรอยู่ในส่วนนั้น และใคร (Who) คือผู้ที่จะต้องสนใจในส่วนนั้นๆ และกล่าวถึงว่าส่วนใดจำเป็นที่ต้องมีในเอกสาร และส่วนใดเป็นส่วนที่เพิ่มเติม อาจจะมีหรือไม่มีส่วนนั้นก็ได้ขึ้นอยู่กับความจำเป็นในการใช้งาน ทำให้การพัฒนาโปรแกรมมีทิศทางที่ค่อนข้างชัดเจน แต่สามารถเพิ่มขยายได้ในอนาคตอีกด้วย

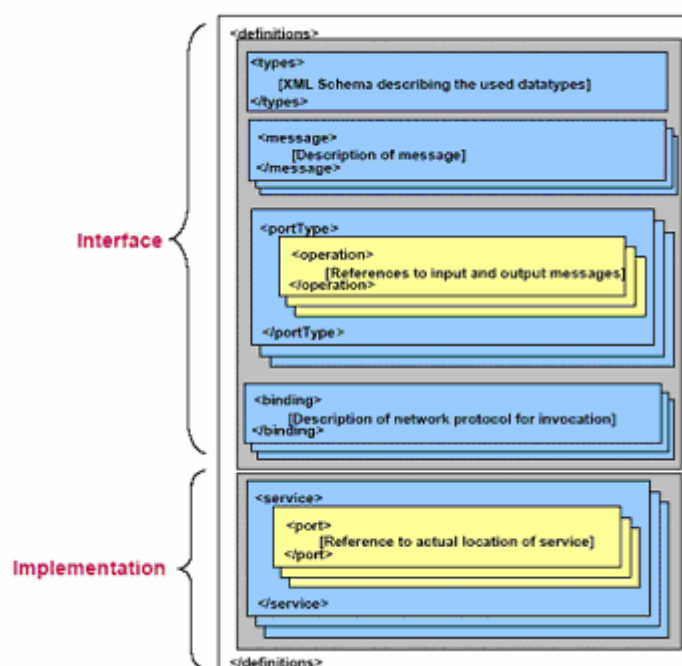
## 2.5. WSDL (Web Services Description Language)

WSDL (Web Services Description Language) เป็นภาษาที่ใช้อธิบายคุณลักษณะการใช้บริการของ Web Services และวิธีการติดต่อกับ Web Services ความต้องการของนิยามนี้เกี่ยวข้องกับความต้องการของ distributed system ที่จะกำหนด Interface Definition Language (IDL) โดยใช้ภาษา XML

WSDL คือ มาตรฐานสำหรับการประกาศ process ที่จำเป็นในการเรียกใช้เซอร์วิส SOAP (Simple Object Access Potocal)

### 2.5.1. โครงสร้างเอกสาร WSDL

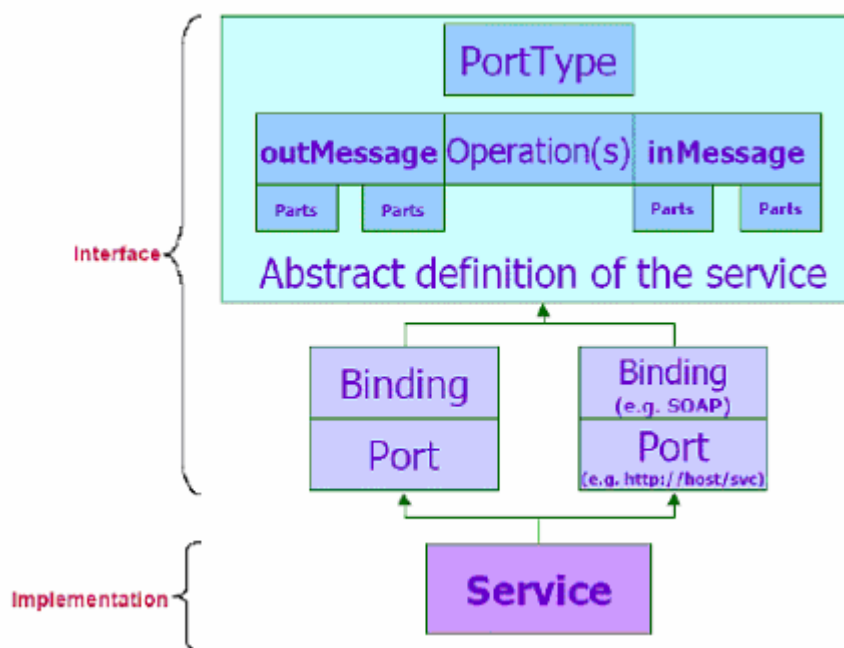
WSDL เป็นภาษาที่อยู่ในความดูแลขององค์กร W3C (World Wide Web Consortium) version ที่มีอยู่ในปัจจุบัน คือ WSDL 1.1 ในการใช้งานจริง หากเราสร้างบริการ Web Services ก็จะมีเครื่องมือช่วยสร้างเอกสาร WSDL สำหรับ Web Services อย่างอัตโนมัติ



รูปที่ 2.10 แสดง ส่วนประกอบของ WSDL ในรูปแบบ XML

WSDL( Web Services Description Language ) ใช้กำหนดเว็บเซอร์วิสหรือ อธิบายในรูปแบบพื้นฐานของ XML schema ไว้สำหรับ

- interfaces
- operation type
- ข้อมูลที่ถูกใช้ สำหรับ เว็บเซอร์วิส
- Data type(XML schema)
- Access protocol เช่น SOAP บน HTTP
- Contact endpoints เช่น บริการบน URL(s)



รูปที่ 2.11 แสดง ส่วนประกอบ WSDL ในรูปแบบโครงสร้าง

#### ส่วนประกอบของ WSDL

- แต่ละ message จะเป็นลำดับรายการในส่วนของ Parts
- Part ง่ายหรือซับซ้อนของ data type ขึ้นอยู่กับการกำหนดที่ดี
- Operation กำหนดระหว่าง การจับคู่ของ input & output ของข้อมูล
- แต่ละ Interface ( portType ) เป็นกลุ่มของ related operations
- Binding เกี่ยวข้องกับ interface ( portType ) ด้วยข้อมูลของ transport protocol
- แต่ละ port มีความเกี่ยวข้องกับ binding
- Service อาจจะใช้ได้ ที่หนึ่งหรือมากกว่า endpoint (ports)

ดูภายในเอกสารที่เราควรรู้เกี่ยวกับการติดต่อและเรียกใช้บริการของ Web Services มีจุดที่ควรรู้ ดังนี้

ตารางที่ 2.5 แสดง WSDL ELEMENT

| Element          | Definition                                                                                                                                                                          |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <types><br>①     | อธิบายชนิดข้อมูลที่ web service ใช้ เพื่อความเป็นกลาง WSDL ใช้ XML Schema syntax ในการระบุชนิดข้อมูล                                                                                |
| <message><br>②   | อธิบาย data elements ของ operation แต่ละ message อาจมีมากกว่าหนึ่งส่วนเทียบได้กับ parameter ของ function ในการเขียนโปรแกรม                                                          |
| <portType><br>③  | เป็นส่วนที่สำคัญที่สุดใน WSDL element อธิบาย operations ที่ web service มีให้บริการและ messages ที่เกี่ยวข้อง เทียบได้กับ function library หรือ module หรือ class ในการเขียนโปรแกรม |
| <binding><br>④   | อธิบาย format ของ message และ protocol details ในแต่ละ port                                                                                                                         |
| <service><br>⑤   | สำหรับ web server จะมี Web Services จำนวนกี่บริการก็ได้ และ ชื่อ Web Services ก็เป็นตัวจำแนกและบ่งบอกแต่ละบริการซึ่งห้ามมีชื่อซ้ำกัน                                                |
| <operation><br>⑥ | อธิบาย method ที่ให้บริการ Web Services หนึ่งจะมี method จำนวนกี่ method ก็ได้                                                                                                      |

ตามทฤษฎีแล้ว ไฟล์เอกสาร WSDL แต่ละไฟล์ สามารถอธิบายคุณลักษณะของบริการ Web Services ได้มากกว่า 1 บริการ โดยแต่ละ Web Services จะมี port สื่อสารเฉพาะตัว ซึ่งบ่งบอกไว้ในเอกสาร WSDL อยู่แล้ว

### 2.5.2. WSDL Example

```
<wsdl:definitions targetNamespace="http://WS" xmlns:impl="http://WS"
xmlns:intf="http://WS" xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:wsdlsoap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

```
<wsdl:types>
<schema elementFormDefault="qualified" targetNamespace="http://WS"
xmlns="http://www.w3.org/2001/XMLSchema" xmlns:impl="http://WS"
```



```

xmlns:intf="http://WS" xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <element name="getCourseAll">
    <complexType>
      <sequence/>
    </complexType>
  </element>
  <element name="getCourseAllResponse">
    <complexType>
      <sequence>
        <element name="getCourseAllReturn" nillable="true"
type="xsd:string"/>
      </sequence>
    </complexType>
  </element>
  <element name="buyClass">
    <complexType>
      <sequence>
        <element name="ClassID" type="xsd:int"/>
        <element name="Email" nillable="true" type="xsd:string"/>
        <element name="ReceiveID" nillable="true"
type="xsd:string"/>
      </sequence>
    </complexType>
  </element>
  <element name="buyClassResponse">
    <complexType>
      <sequence>
        <element name="buyClassReturn" type="xsd:boolean"/>
      </sequence>
    </complexType>
  </element>
  <element name="reservation">
    <complexType>
      <sequence>
        <element name="ClassID" type="xsd:int"/>
        <element name="Email" nillable="true" type="xsd:string"/>
        <element name="Name" nillable="true" type="xsd:string"/>
        <element name="Surname" nillable="true" type="xsd:string"/>
        <element name="PhoneNo" nillable="true" type="xsd:string"/>
      </sequence>
    </complexType>
  </element>
  <element name="reservationResponse">
    <complexType>
      <sequence>
        <element name="reservationReturn" type="xsd:boolean"/>
      </sequence>
    </complexType>
  </element>

```

```

<element name="getCourse">
  <complexType>
    <sequence>
      <element name="Subject" nillable="true" type="xsd:string"/>
      <element name="Period" type="xsd:int"/>
    </sequence>
  </complexType>
</element>
<element name="getCourseResponse">
  <complexType>
    <sequence>
      <element name="getCourseReturn" nillable="true" type="xsd:string"/>
    </sequence>
  </complexType>
</element>
<element name="getCourseFromEmail">
  <complexType>
    <sequence>
      <element name="Email" nillable="true" type="xsd:string"/>
    </sequence>
  </complexType>
</element>
<element name="getCourseFromEmailResponse">
  <complexType>
    <sequence>
      <element name="getCourseFromEmailReturn" nillable="true"
        type="xsd:string"/>
    </sequence>
  </complexType>
</element>
<element name="cancel">
  <complexType>
    <sequence>
      <element name="ClassID" type="xsd:int"/>
      <element name="Email" nillable="true" type="xsd:string"/>
    </sequence>
  </complexType>
</element>
<element name="cancelResponse">
  <complexType>
    <sequence>
      <element name="cancelReturn" type="xsd:boolean"/>
    </sequence>
  </complexType>
</element>
</schema>
</wsdl:types>

```

```

<wsdl:message name="getCourseFromEmailRequest">
  <wsdl:part element="intf:getCourseFromEmail" name="parameters"/>

```

```

</wsdl:message>
<wsdl:message name="buyClassResponse">
  <wsdl:part element="intf:buyClassResponse" name="parameters"/>
</wsdl:message>
<wsdl:message name="getCourseFromEmailResponse">
  <wsdl:part element="intf:getCourseFromEmailResponse"
name="parameters"/>
</wsdl:message>
<wsdl:message name="getCourseAllResponse">
  <wsdl:part element="intf:getCourseAllResponse" name="parameters"/>
</wsdl:message>
<wsdl:message name="getCourseAllRequest">
  <wsdl:part element="intf:getCourseAll" name="parameters"/>
</wsdl:message>
<wsdl:message name="cancelRequest">
  <wsdl:part element="intf:cancel" name="parameters"/>
</wsdl:message>
<wsdl:message name="reservationRequest">
  <wsdl:part element="intf:reservation" name="parameters"/>
</wsdl:message>
<wsdl:message name="getCourseRequest">
  <wsdl:part element="intf:getCourse" name="parameters"/>
</wsdl:message>
<wsdl:message name="getCourseResponse">
  <wsdl:part element="intf:getCourseResponse" name="parameters"/>
</wsdl:message>
<wsdl:message name="buyClassRequest">
  <wsdl:part element="intf:buyClass" name="parameters"/>
</wsdl:message>
<wsdl:message name="reservationResponse">
  <wsdl:part element="intf:reservationResponse" name="parameters"/>
</wsdl:message>
<wsdl:message name="cancelResponse">
  <wsdl:part element="intf:cancelResponse" name="parameters"/>
</wsdl:message>

```

```

<wsdl:portType name="ApplyPhysic">
  <wsdl:operation name="getCourseAll">
    <wsdl:input message="intf:getCourseAllRequest"
name="getCourseAllRequest"/>
    <wsdl:output message="intf:getCourseAllResponse"
name="getCourseAllResponse"/>
  </wsdl:operation>
  <wsdl:operation name="buyClass">
    <wsdl:input message="intf:buyClassRequest" name="buyClassRequest"/>
    <wsdl:output message="intf:buyClassResponse" name="buyClassResponse"/>
  </wsdl:operation>
  <wsdl:operation name="reservation">
    <wsdl:input message="intf:reservationRequest"
name="reservationRequest"/>

```

```

        <wsdl:output message="intf:reservationResponse" name="reservationResponse"/>
    </wsdl:operation>
    <wsdl:operation name="getCourse">
        <wsdl:input message="intf:getCourseRequest"
name="getCourseRequest"/>
        <wsdl:output message="intf:getCourseResponse" name="getCourseResponse"/>
    </wsdl:operation>
    <wsdl:operation name="getCourseFromEmail">
        <wsdl:input message="intf:getCourseFromEmailRequest"
name="getCourseFromEmailRequest"/>
        <wsdl:output message="intf:getCourseFromEmailResponse"
name="getCourseFromEmailResponse"/>
    </wsdl:operation>
    <wsdl:operation name="cancel">
        <wsdl:input message="intf:cancelRequest" name="cancelRequest"/>
        <wsdl:output message="intf:cancelResponse" name="cancelResponse"/>
    </wsdl:operation>
</wsdl:portType>

```

6

```

<wsdl:binding name="ApplyPhysicSoapBinding" type="intf:ApplyPhysic">
    <wsdlsoap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
    <wsdl:operation name="getCourseAll">
        <wsdlsoap:operation soapAction=""/>
        <wsdl:input name="getCourseAllRequest">
            <wsdlsoap:body use="literal"/>
        </wsdl:input>
        <wsdl:output name="getCourseAllResponse">
            <wsdlsoap:body use="literal"/>
        </wsdl:output>
    </wsdl:operation>
    <wsdl:operation name="buyClass">
        <wsdlsoap:operation soapAction=""/>
        <wsdl:input name="buyClassRequest">
            <wsdlsoap:body use="literal"/>
        </wsdl:input>
        <wsdl:output name="buyClassResponse">
            <wsdlsoap:body use="literal"/>
        </wsdl:output>
    </wsdl:operation>
    <wsdl:operation name="reservation">
        <wsdlsoap:operation soapAction=""/>
        <wsdl:input name="reservationRequest">
            <wsdlsoap:body use="literal"/>
        </wsdl:input>
        <wsdl:output name="reservationResponse">
            <wsdlsoap:body use="literal"/>
        </wsdl:output>
    </wsdl:operation>
    <wsdl:operation name="getCourse">
        <wsdlsoap:operation soapAction=""/>

```

4

```

    <wsdl:input name="getCourseRequest">
      <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="getCourseResponse">
      <wsdlsoap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="getCourseFromEmail">
    <wsdlsoap:operation soapAction=""/>
    <wsdl:input name="getCourseFromEmailRequest">
      <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="getCourseFromEmailResponse">
      <wsdlsoap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="cancel">
    <wsdlsoap:operation soapAction=""/>
    <wsdl:input name="cancelRequest">
      <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="cancelResponse">
      <wsdlsoap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
</wsdl:binding>

```

```

    <wsdl:service name="ApplyPhysicService">
      <wsdl:port binding="intf:ApplyPhysicSoapBinding"
name="ApplyPhysic">
        <wsdlsoap:address
          location="http://161.246.6.91:9080/ApplyPhysic/services/ApplyPhysic"
        </>
      </wsdl:port>
    </wsdl:service>
</wsdl:definitions>

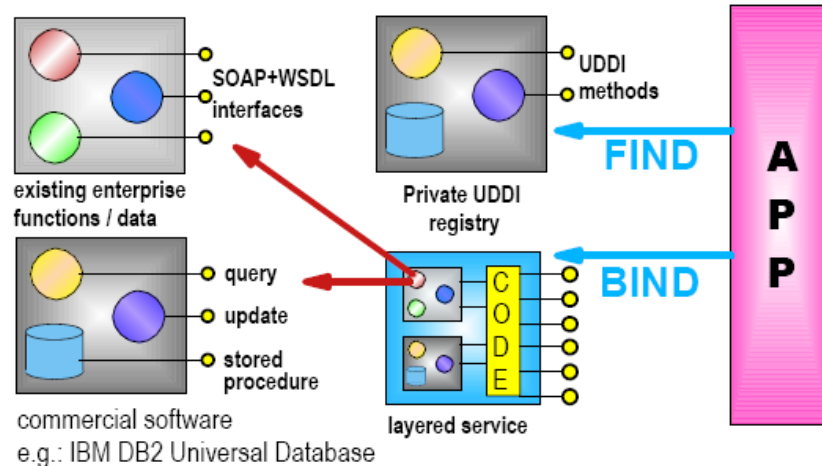
```

## 2.6. UDDI (Universal Description, Discovery and Integration protocol)

UDDI (Universal Description, Discovery, and Integration) เป็นมาตรฐานที่ให้ชุดพื้นฐาน APIs (Application Programming Interface) ของ SOAP ที่สามารถนำมาใช้ในการพัฒนาเป็นตัวแทนของผู้ให้บริการ (Service broker) UDDI ใช้สำหรับค้นหา Service ที่ต้องการและเมื่อได้มาแล้ว UDDI ยังจัดหาข้อมูลลงในวิธีการที่จะใช้งานเปรียบได้กับสมุดหน้าเหลือง เป็นมาตรฐานที่จัดตั้งขึ้นโดยบริษัทไอบีเอ็ม บริษัทไมโครซอฟต์ และบริษัทอริบา (Ariba) ปัจจุบันมีบริษัทที่ร่วมกันกำหนดมาตรฐานของ UDDI มากกว่า 70 บริษัท ซึ่งมาตรฐานของ UDDI ถูกกำหนดให้เป็นมาตรฐานสำหรับ B2B interoperability

## UDDI for EAI Use

- SOAP+WSDL is a useful strategy for application or data integration with loose coupling
  - ▶ Especially helpful in mergers and acquisitions
  - ▶ Often fits into existing architectures, making them more flexible and adaptable
- Private UDDI can organize services in your enterprise



รูปที่ 2.12 แสดงการทำงานของ UDDI